
一维数组的常用操作

复习

1. 前言
2. 一维数组的定义和初始化 重点
3. 一维数组数据元素的访问 重点
4. 一维数组的应用案例-冒泡排序 熟悉
5. 断点调试 重点

目录

- | | |
|-------------|----|
| 1. 一维数组查找元素 | 重点 |
| 2. 一维数组修改元素 | 重点 |
| 3. 一维数组添加元素 | 重点 |
| 4. 一维数组删除元素 | 重点 |
| 5. 一维数组合并 | 重点 |

一维数组查找元素

普通查找

二分查找

普通查找

根据值顺序查找

比如在给定数组中查找固定的某个值，就可以从下标为0的元素开始顺序对比查找。

普通查找 案例

在长度为N ($N \leq 100$)的数组中，输入m个元素，查找k元素第一次出现的位置(位置从1开始)，如果找不到k，输出-1.

输入案例：

5

1 3 5 6 3

3

输出案例：

2

普通查找 代码

```
const int N = 100 ; //数组长度
int arr[N],m,k,index = -1;
cin >> m;
for(int i=0; i<m; i++){
    cin >> arr[i];
}
cin >> k;
for(int i=0; i<m;i++){ //顺序查找
    if(k == arr[i]){
        index = i + 1;
        break;
    }
}
cout << index << endl;
```

二分查找

二分查找也称折半查找(Binary Search)，是一种效率较高的查找方法。

猜数字游戏？

为啥先猜50？

二分查找 逻辑

思路同猜数字大小：

- 1.定义两个变量l和r；
- 2.用mid（猜的数）保存l和r的中间值；
- 3.假如小了，则把l变成mid+1（去除左半部）；
- 4.假如大了，则把r变成mid-1（去除右半部）；
- 5.假如对了，则返回mid；
- 6.假如 $l > r$ ，则返回-1；
- 7.重复第2-6句

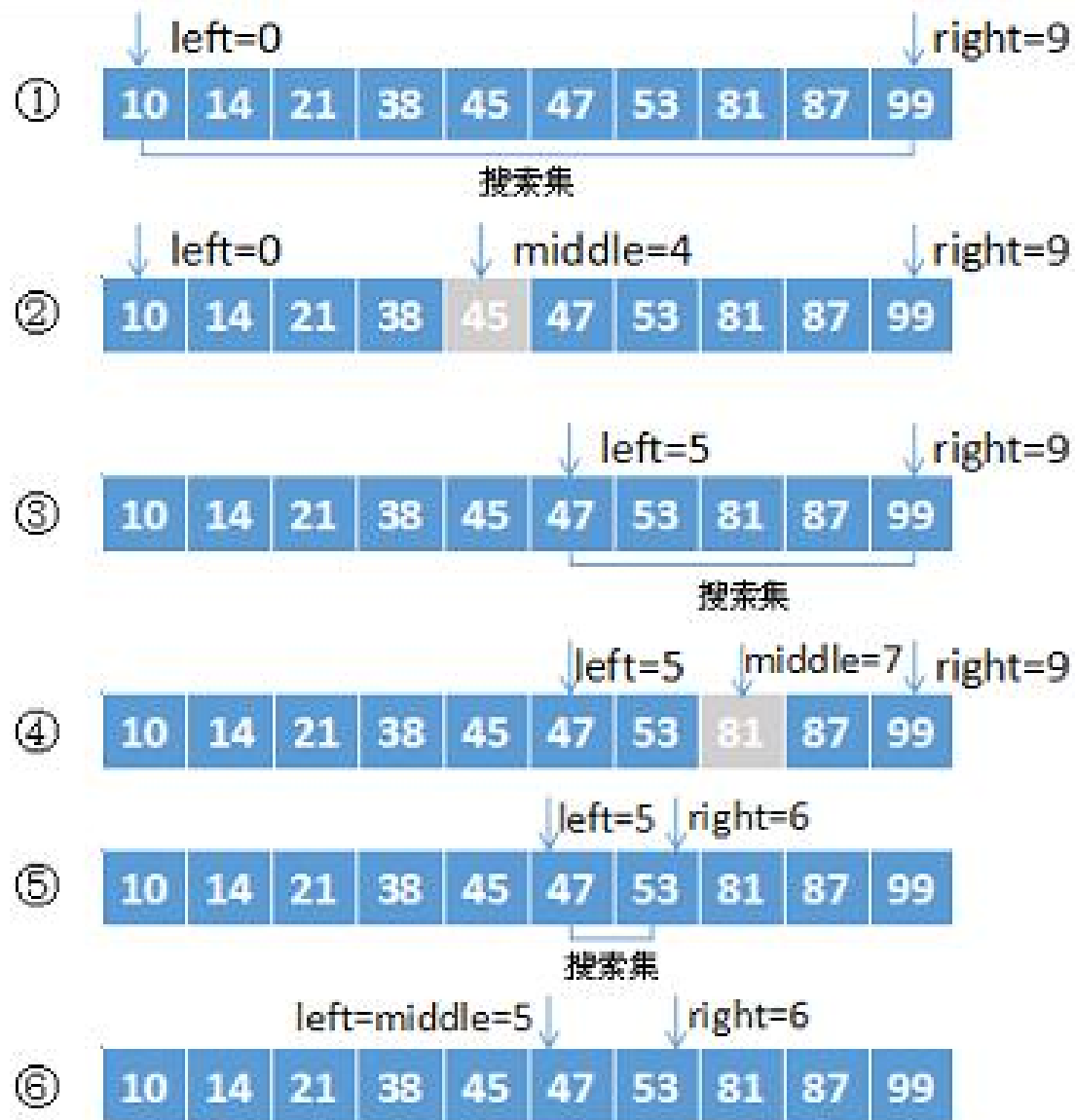
二分查找

注意点：

- 1.用于查找的内容逻辑上来说是需要有序的
- 2.查找的数量只能是一个，而不是多个

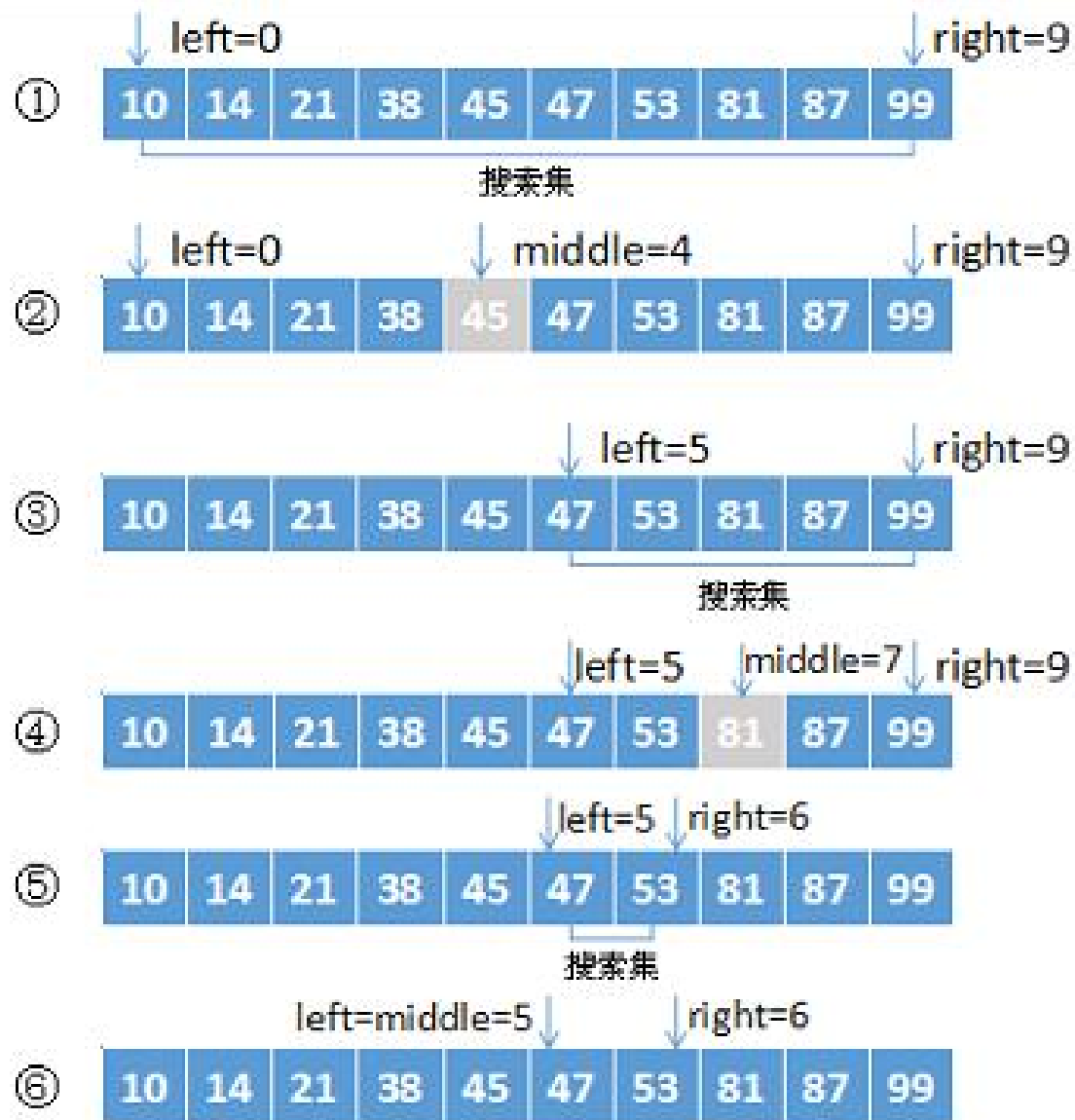
二分查找图解

查找47



二分查找图解

查找51



找不到会怎样？

二分查找 案例

在数组长度为N ($N \leq 100$)的数组中，输入m个元素（有序），查找k元素的位置(位置从1开始)，如果找不到k，输出-1.

案例输入：

5

11 22 33 44 55

22

案例输出：

2

二分查找 代码

```
int const N = 100; //数组长度
int arr[N]; //声明数组
int m, k; //m元素个数，k查找的元素
cin >> m;
for (int i = 0; i < m; i++) {
    cin >> arr[i];
}
cin >> k;
//left起始下标，right最大下标，mid保存中间下标
int left = 0, right = m - 1, mid = 0;
while (left <= right) { //当left小于等于right时查找
    mid = (left + right) / 2; //计算中间值mid
    if (k == arr[mid]) //查找到k
    {
        cout << mid + 1 << endl;
        break;
    }
    else if (k < arr[mid]) { //小于的情况，right挪到mid-1
        right = mid - 1;
    }
    else if (k > arr[mid]) { //大于的情况，left挪到mid+1
        left = mid + 1;
    }
}
if (left > right) //当left < right, k元素不在数组中
{
    cout << -1 << endl;
}
```

二分查找 最小下标

如果需要查找的元素不止一个，怎么办？

1 2 **3** 3 3 3 3 3 3 5 6

最小下标案例

在数组长度为N ($N \leq 100$)的数组中，输入m个元素（有序），查找k元素的最小位置(位置从1开始)，如果找不到k，输出-1.

案例输入：

10

11 22 22 22 22 22 22 33 44 55

22

案例输出：

2

最小下标 代码

```
if (k == arr[mid]) //查找到k
{
    int index = mid;
    //使用while往左查找最小下标
    while (index >= 0 && arr[index] == k) {
        index--;
    }
    cout << index + 1 << endl;
    break;
}
```

二分查找 最大下标

如果需要查找的元素不止一个，怎么办？

1 2 3 3 3 3 3 3 **3** 5 6

最大下标案例

在数组长度为N ($N \leq 100$)的数组中，输入m个元素（有序），查找k元素的最小位置(位置从1开始)，如果找不到k，输出-1.

案例输入：

10

11 22 22 22 22 22 22 33 44 55

22

案例输出：

7

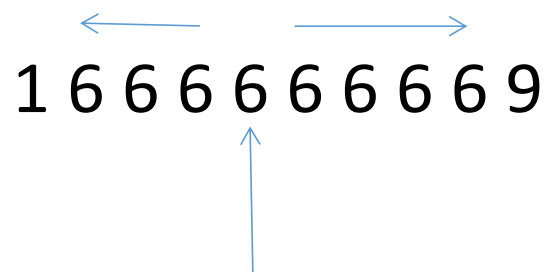
最大下标 代码

```
if (k == arr[mid]) //查找到k
{
    int index = mid;
    //使用while往右查找最大下标
    while (index <= m - 1 && arr[index] == k) {
        index++;
    }
    cout << index + 1 << endl;
    break;
}
```

查找元素个数

在一个有序数组中，相同元素一定是挨着的，所以，只要找到这个元素，在左右寻找，记录个数即可。

1 6 6 6 6 6 6 6 6 9



8个

查找元素个数案例

在元素个数 N ($N \leq 100$)的数组中，输入 m 个元素（有序），查找 k 元素的个数，如果找不到，输出0.

案例输入：

10

11 22 22 22 22 22 22 33 44 55

22

案例输出：

6

查找元素个数代码

```
int const N = 100; //数组长度
int arr[N]; //声明数组
int m, k; //m元素个数，k查找的元素
cin >> m;
for (int i = 0; i < m; i++) {
    cin >> arr[i];
}
cin >> k;
//left起始下标，right最大下标，mid保存中间下标
int left = 0, right = m - 1, mid = 0, count = 0;
while (left <= right) { //当left小于等于right时查找
    mid = (left + right) / 2; //计算中间值mid
    if (k == arr[mid]) //查找到k
    {
        int index1 = mid-1, index2 = mid+1;
        count = 1;
        while (index1 >= 0 && arr[index1] == k) { //扫描左边
            index1--;
            count++;
        }
        while (index2 <= m-1 && arr[index2] == k) { //扫描右边
            index2++;
            count++;
        }
        break;
    }
    else if (k < arr[mid]) { //小于的情况，right挪到mid-1
        right = mid - 1;
    }
    else if (k > arr[mid]) { //大于的情况，left挪到mid+1
        left = mid + 1;
    }
}
cout << count << endl;
```

一维数组修改元素

- 1.通过下标直接修改
- 2.先查找，在修改

通过下标直接修改

找到对应下标修改

比如修改下标为偶数的值，或者修改下标为奇数的值

再或者修改下标能够被3整除的元素等等

通过下标直接修改 案例

蛋糕厂有两台包装称重机编号01与02,以01号包装一块、02号包装一块..这样的方式轮流工作。经检查今日02号机器出现故障,每一块小蛋糕都少称m克,已知今日包装了n块小蛋糕,输入修正前n块小蛋糕的重量,输出修正后每块小蛋糕的重量。

输入格式:两行,第一行一个整数n($0 < n \leq 10000$),第二行为n个浮点数表示修正前n个小蛋糕的质量,第三行一个浮点数m表示因机器故障导致每块小蛋糕缺少质量。

输出格式:一行,修正后的小蛋糕重量。

输入用例:

5

50.5 48.2 50.5 47.7 50.2

3.2

输出用例:

50.5 51.4 50.5 50.9 50.2

通过下标直接修改 代码

```
double arr[10000],m;
int n;
cin >> n;
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
cin >> m;

for (int i = 0; i < n; i++)
{
    if (i % 2 == 1) {
        arr[i] += m;
    }
}
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
```

先查找，在修改

根据要求找到对应下标，在进行修改

先查找，在修改 案例

在一次100百米跑比赛中，有 n ($5 \leq n \leq 10$)个同学进行比赛，输入这 n 个同学的比赛成绩，把第一名和最后一名的同学的位置进行交换，在输出比赛成绩。

输入用例:

5

10.3 9.8 10.6 9.9 11

输出用例:

10.3 11 10.6 9.9 9.8

先查找，在修改 代码

```
double arrScore[20];
int n;
cin >> n;
for (int i = 0; i < n; i++)
{
    cin >> arrScore[i];
}

double first = arrScore[0], last = arrScore[0];
int first_index = 0, last_index = 0;
for (int i = 0; i < n; i++) {
    if (arrScore[i] < first) {
        first = arrScore[i];
        first_index = i;
    }
    if (arrScore[i] > last)
    {
        last = arrScore[i];
        last_index = i;
    }
}
swap(arrScore[first_index], arrScore[last_index]);

for (int i = 0; i < n; i++) {
    cout << arrScore[i] << " ";
}
```

一维数组添加元素

1. 添加到数组末尾
2. 根据要求添加到数组

添加到数组末尾

根据现有最大下标加1，进行插入

注意：插入的下标一定小于等于数组长度减1

添加到数组末尾 案例

马路前方塞车，目前塞车数量 n ，输入 n 辆车编号，后来又有 m 辆加入塞车队伍，输出现在所有塞车车辆编号，总车辆 ($n+m \leq 100$).

输入用例：

3

12 15 9

5

10 11 8 19 6

输出用例：

12 15 9 10 11 8 19 6

添加到数组末尾 代码

```
int arr[1000], n, m;  
cin >> n;  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
cin >> m;  
for (int i = n; i < (n + m); i++) {  
    cin >> arr[i];  
}  
  
for (int i = 0; i < (n + m); i++) {  
    cout << arr[i] << " ";  
}
```

根据要求添加到数组

根据要求有可能在数组头部添加，也有可能是在数组中间添加。

根据要求添加到数组 案例

班里转来一位新同学,班主任老师安排这位同学坐在小张这一组,输出安排好新同学座位后小张所在小组每位同学的学号。

输入格式:四行,第一行为一个整数 n ($0 < n < 60$)代表小张同学这一组的同学人数,第二行为 n 个整数代表小张同学所在小组每位学生的学号,第三行为一个整数 k 代表新来同学的学号,第四行两个整数,第一个整数 p ,第二个整数位 m ,若 p 为1,则新同学坐在第 m 个同学的前面,若 p 为0,则新同学坐在第 m 个同学的后面。

输出格式: 一行,新同学安排好后,小张所在小组每位同学的学号。

输入用例:

5

7 1 3 14 5

56

0 2

输出用例:

7 1 56 3 14 5

根据要求添加到数组 代码

```
int arr[100];
int n, k, p, m;
cin >> n;
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
cin >> k;
cin >> p >> m;
int index = n - 1;
if (p) {
    while (index >= m - 1)
    {
        arr[index + 1] = arr[index];
        index--;
    }
    arr[m-1] = k;
}
else {
    while (index > m - 1)
    {
        arr[index + 1] = arr[index];
        index--;
    }
    arr[m] = k;
}
for (int i = 0; i < (n + 1); i++) {
    cout << arr[i] << " ";
}
```

一维数组删除元素

根据要求删除数组元素

指定下标删除 案例

小赵在录入 n ($2 \leq n \leq 100$)个同学的分数，该数组中有个数据是严重错误的分数，找出该分数，删掉后，打印正常分数。

输入用例：

5

90 98 87 -100 89

输出用例：

90 98 87 89

指定下标删除 代码

```
int n;
int arr[100];
cin >> n;
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
int index = -1;
for (int i = 0; i < n; i++)
{
    if (arr[i] < 0) {
        index = i;
        break;
    }
}
while (index < n - 1)
{
    arr[index] = arr[index + 1];
    index++;
}

for (int i = 0; i < n - 1; i++)
{
    cout << arr[i] << " ";
}
```

一维数组合并

- 1.无序合并
- 2.有序合并

无序合并

根据数组长度，合并成一个数组。

无序合并 案例

给定两个数组arr1和arr2，数组长度不超过100，合并成arr3.

输入用例：

4

11 2 13 41

3

100 200 300

输出用例：

11 2 13 41 100 200 400

无序合并 代码

```
int n1,n2, arr1[100], arr2[100], arr3[200];
cin >> n1;
for (int i = 0; i < n1; i++)
{
    cin >> arr1[i];
}
cin >> n2;
for (int i = 0; i < n2; i++)
{
    cin >> arr2[i];
}

for (int i = 0; i < n1; i++)
{
    arr3[i] = arr1[i];
}
int index = 0;
for (int i = n1; i < n1 + n2; i++)
{
    arr3[i] = arr2[index ++];
}

for (int i = 0; i < n1 + n2; i++)
{
    cout << arr3[i] << " ";
}
```

有序合并

一般合并之后的数组是有序的

有序合并 案例

输入两个升序整数数组nums1和nums2,将nums1与nums2合并到nums3中,并且删除重复的数,使得nums3依旧为一个升序数组。(不能先合并再排序)

输入格式:两行,第一行先输入一个整数N1($0 < N1 < 100$)代表nums1数组的元素个数,再输入N1个整数代表nums1中的N1个元素值。第二行先输入一个整数N2($0 < N2 < 100$)代表nums2数组的元素个数,再输入N2个整数代表num2中的N2个元素值。

输出格式:一行,合并后的nums3数组。

输入用例:

6

-3 2 5 7 8 12

8

0 1 2 3 10 13 18 20

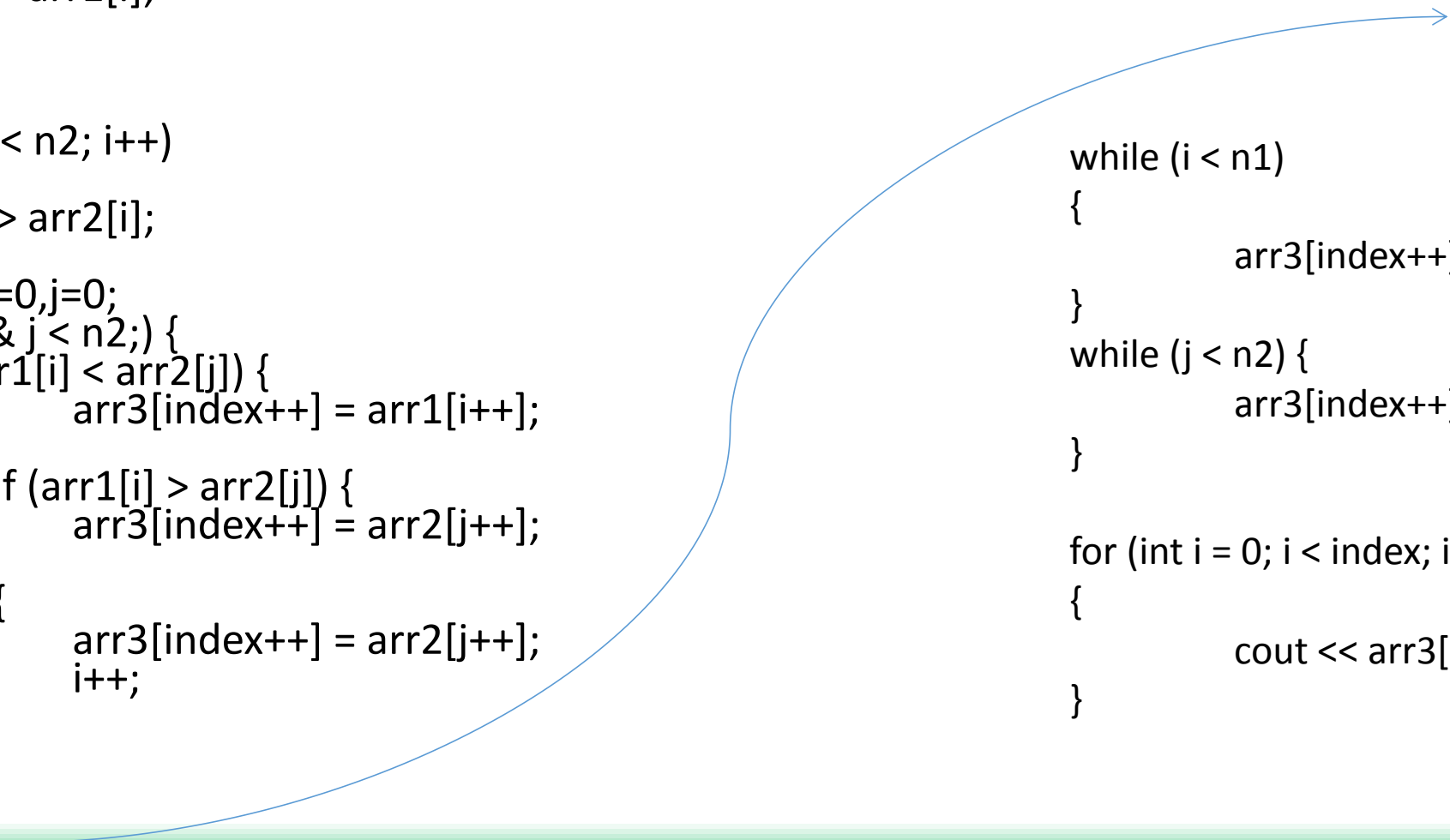
输出用例:

-3 0 1 2 3 5 7 8 10 12 13 18 20

有序合并 代码

```
int n1, n2, arr1[100], arr2[100], arr3[200];
cin >> n1;
for (int i = 0; i < n1; i++)
{
    cin >> arr1[i];
}

cin >> n2;
for (int i = 0; i < n2; i++)
{
    cin >> arr2[i];
}
int index = 0, i=0, j=0;
for (; i < n1 && j < n2;) {
    if (arr1[i] < arr2[j]) {
        arr3[index++] = arr1[i++];
    }
    else if (arr1[i] > arr2[j]) {
        arr3[index++] = arr2[j++];
    }
    else {
        arr3[index++] = arr2[j++];
        i++;
    }
}
```



```
while (i < n1)
{
    arr3[index++] = arr1[i++];
}
while (j < n2) {
    arr3[index++] = arr2[j++];
}

for (int i = 0; i < index; i++)
{
    cout << arr3[i] << " ";
}
```

课堂练习

定义一个int型的一维数组，包含10个元素，分别赋值为1~10, .然后将数组中的元素都向前移一个位置，即 $a[0]=a[1]$, $a[1]=a[2]$...最后一个元素的值是原来第一个元素的值，然后输出这个数组。

输入：

1 2 3 4 5 6 7 8 9 10

输出：

2 3 4 5 6 7 8 9 10 1

课堂练习代码

```
int arr[10];
for (int i = 0; i < 10; i++)
{
    cin >> arr[i];
}
int temp = arr[0];
for (int i = 0; i < 9; i++)
{
    arr[i] = arr[i + 1];
}
arr[9] = temp;
for (int i = 0; i < 10; i++)
{
    cout << arr[i] << " ";
}
```

知识点总结

- | | |
|-------------|----|
| 1. 一维数组查找元素 | 重点 |
| 2. 一维数组修改元素 | 重点 |
| 3. 一维数组添加元素 | 重点 |
| 4. 一维数组删除元素 | 重点 |
| 5. 一维数组合并 | 重点 |

作业

输入n个从小到大的整数序列，把整数m插入其中，整个序列仍然保持升序。

输入用例：

5

1 5 8 11 15

3

输出用例：

1 3 5 8 11 15

下节课预告

char数组